Logging

Recording what a program does, and doing it safely

This part replaces debugging print statements with *logging*: messages that have levels, that can be turned up or down without editing the code, and that can go to the console, a file, or both. It covers Java's built-in logger, the logging libraries used in industry, and a question that comes up often: is Java logging a security risk? For reading, see Core Java, Volume I, Chapter 7.

A note before you begin. These are my notes from over the years, cleaned up so they are readable. They are not a study guide, and they are not gospel: not everything here appears in a lecture, and not every topic has notes yet. I write from memory and revise every semester. I also use AI to help convert documents to LaTeX, check for issues, and add explanations. I do not trust it completely, so I am still editing, and a typo or imprecise statement can survive longer than it should. Java also changes over time, though the basics we cover are resilient. If you find something that looks wrong, contradicts the book, or does not behave the way I claim when you run it, tell me. You will be doing the other students a favor, and you will understand the material better for having caught it. The textbook is the tiebreaker, and the compiler beats us both. These notes sometimes go beyond the book, but they do not replace the assigned chapters or the recorded lectures, and neither of those replaces the notes. Read the chapter, watch the lecture, and study the notes.

1. Why Log?

Every program so far has reported its progress with `System.out.println`. That works for small programs, but it has real problems.

- It mixes messages meant for the programmer into the program's real output. That is why the labs list "No unrequested extra output" as a requirement.
- It cannot be turned off without editing the code, so debugging lines are either deleted, and lost, or left in to clutter the output.
- It has no sense of importance. A routine "loading the file" message looks the same as "the payroll could not be saved."
- It does not record when something happened or where in the code it came from.

A *logger* solves all four. Every message has a *level*, the output can be filtered by level without changing the code, and each message is stamped with the time and its source. Logs can go to the console, to a file, or both.

2. java.util.logging

Java includes a logging framework in the package `java.util.logging`. Each class usually gets its own logger, named after the class:

```
public class LogDemo {
    private static final Logger LOG = Logger.getLogger(LogDemo.class.getName());
```

```
    public static void main(String[] args) {
        String fileName = "files/staff.csv";
        LOG.info("Loading " + fileName);
        LOG.fine("This FINE message is hidden by default");
        try {
            Files.readString(Path.of(fileName));
        } catch (IOException e) {
            LOG.log(Level.SEVERE, "Could not read " + fileName, e);
        }
        LOG.log(Level.WARNING, "Skipped line {0}: {1}", new Object[] {2, "salary is not
     a number"});
    }
}
```

When the file is missing, the output looks like this. The stack trace is shortened here.

```
Oct 24, 2026 6:33:45 PM LogDemo main
INFO: Loading files/staff.csv
Oct 24, 2026 6:33:45 PM LogDemo main
SEVERE: Could not read files/staff.csv
java.nio.file.NoSuchFileException: files/staff.csv
        at java.base/sun.nio.fs.UnixException.translateToIOException(UnixException.java
    :92)
        ...
        at LogDemo.main(LogDemo.java:15)

Oct 24, 2026 6:33:46 PM LogDemo main
WARNING: Skipped line 2: salary is not a number
```

A few things to notice:

- Each message shows the time, the class, and the method that logged it.
- The `FINE` message did not appear. By default, only `INFO` and above are shown.
- Passing the exception as the last argument to `log` records its full stack trace, including any "Caused by" section from Part 2. This is where stack traces belong in a finished program, rather than in `e.printStackTrace()`.
- `{0}` and `{1}` are placeholders, filled from the array in order.
- Log messages go to the standard error stream, not standard output, so they stay separate from the program's real output. Eclipse shows standard error in red.

Levels

Table 1: The java.util.logging levels, from most to least serious.

Level	Use it for
`SEVERE`	A failure: something the program needed to do did not happen.
`WARNING`	A problem the program worked around, such as a skipped record.
`INFO`	Normal milestones, such as a file loaded or a report written. The default threshold.
`CONFIG`	Settings the program is running with.
`FINE`, `FINER`, `FINEST`	Increasingly detailed tracing for debugging.

A shorter format

The default two-line format is noisy. The `java.util.logging.SimpleFormatter.format` system property changes it. In Eclipse, open **Run**, then **Run Configurations**, select your program, and add this under **VM arguments** on the **Arguments** tab. That is the same place you turn on assertions with `-ea` (Part 3):

```
-Djava.util.logging.SimpleFormatter.format="%4$s: %5$s%6$s%n"
```

In that pattern, `%4$s` is the level, `%5$s` the message, `%6$s` the stack trace if there is one, and `%n` a line break. The same program now logs:

```
INFO: Loading files/staff.csv
SEVERE: Could not read files/staff.csv
java.nio.file.NoSuchFileException: files/staff.csv
        ...
WARNING: Skipped line 2: salary is not a number
```

The remaining examples use this format.

3. Handlers: Logging to a File

A logger hands each message to one or more *handlers*, which decide where it goes. By default there is one handler, which writes to the console. Adding a `FileHandler` sends messages to a file as well:

```
public class FileLogDemo {
    private static final Logger LOG = Logger.getLogger(FileLogDemo.class.getName());

    public static void main(String[] args) throws IOException {
        FileHandler handler = new FileHandler("files/payroll.log");
        handler.setFormatter(new SimpleFormatter());   // the default is XML
        handler.setLevel(Level.FINE);
        LOG.addHandler(handler);
        LOG.setLevel(Level.FINE);
```

```
        LOG.info("Payroll run started");
        LOG.fine("Read 5 lines");
        LOG.warning("Skipped 3 bad lines");
        handler.close();
    }
}
```

The console shows:

```
INFO: Payroll run started
WARNING: Skipped 3 bad lines
```

`files/payroll.log` **holds:**

```
INFO: Payroll run started
FINE: Read 5 lines
WARNING: Skipped 3 bad lines
```

The `FINE` message reached the file but not the console. Both the logger and each handler have a level, and a message must pass both. The logger was set to `FINE`, the file handler to `FINE`, and the console handler kept its default of `INFO`. That is the usual arrangement: a quiet console for the user, and a detailed file for the programmer.

Two details worth knowing: without the `setFormatter` line, a `FileHandler` writes XML, and the handler should be closed when the program is done with it, like any file. Settings like these can also go in a `logging.properties` file instead of in code, so they can change without recompiling.

4. System.Logger and Other Frameworks

Since Java 9, the platform also has a small logging interface, `System.Logger`, added by JEP 264. It does not replace `java.util.logging`. It is a front end that sends messages to whatever logging framework the application uses, and by default that is `java.util.logging`:

```
private static final System.Logger LOG = System.getLogger(SysLogDemo.class.getName());
...
LOG.log(System.Logger.Level.INFO, "Loading {0}", "files/staff.csv");
LOG.log(System.Logger.Level.WARNING, "Skipped line {0}", 2);
```

```
INFO: Loading files/staff.csv
WARNING: Skipped line 2
```

Outside the JDK, most professional Java projects use a separate logging library. The common ones are SLF4J, which is a front end much like `System.Logger`, and Logback and Log4j 2, which do the actual logging. The ideas you have learned here, which are loggers named by class, levels, handlers (often called appenders), and formats, carry over to all of them. In this course we use `java.util.logging`, because it is built in and needs no setup.

5. Is Java Logging a Security Risk?

You may have heard that it is. The story behind that is real, but it is about a specific library, not about logging in general or about `java.util.logging`.

Log4Shell

In December 2021, a flaw was disclosed in **Apache Log4j 2**, the widely used third-party logging library mentioned above. It became known as *Log4Shell* (CVE-2021-44228), and it received the highest possible severity score, 10.0 out of 10.

Log4j 2 had a feature called *lookups*: text such as `${...}` inside a message was replaced with a value before the message was written. One kind of lookup, `jndi`, could contact a server over the network. So if a program logged anything an attacker controlled, such as a user name or a web browser's identification string, the attacker could put a lookup in it. Logging that text made the server reach out to the attacker's machine and run code from it. The attacker never needed a password; they only needed their text to be logged.

The Apache project fixed the flaw in version 2.15.0, followed by further fixes in later releases, and the advice was to update immediately. The JDK itself was not the problem, and `java.util.logging` has no lookup feature, so programs that used only the built-in logger were not affected by Log4Shell.

What Log4Shell teaches about all logging

The deeper lesson applies to every logging framework, including the one in these notes: *a logger processes untrusted input.* Three habits follow from that.

1. Do not let input forge log entries. This is called *log injection.* If a user name contains a line break, the user can make the log appear to contain an entry that never happened:

```
static String clean(String text) {
    return text.replaceAll("[\\r\\n]", "_");
}
...
String user = "guest\nINFO: Login succeeded for administrator";
LOG.warning("Login failed for " + user);
LOG.warning("Login failed for " + clean(user));
```

```
WARNING: Login failed for guest
INFO: Login succeeded for administrator
WARNING: Login failed for guest_INFO: Login succeeded for administrator
```

The first message produced two lines, and the second one is a lie that a person reading the log would believe. The cleaned version replaces the line break, so the forged text stays on the same line as the real message, where it is obviously part of the user name. Validating input before it is used at all, as in Part 2, is even better.

2. Do not log secrets. Logs are often kept for a long time, copied to other systems, and read by people who should not see sensitive data. Never log passwords, access tokens, session IDs, encryption keys, database connection strings with passwords, payment card or bank numbers, or personal information you do not need. Log that a login failed, not the password that was tried.

3. Keep libraries up to date. Log4Shell did its damage because the vulnerable library was inside thousands of programs whose authors did not know it was there or did not update it. A library you add to a project is code you are responsible for, including its security fixes.

So the accurate answer is: logging is not unsafe, but a logger is part of your program's attack surface. Treat what you log as untrusted data, keep secrets out of it, and keep your logging library current.

6. Summary

Table 2: Logging in brief.

Tool	Key points
`Logger`	One per class, named after the class. Methods by level: `severe`, `warning`, `info`, `fine`, and `log`.
Levels	`SEVERE` down to `FINEST`. The console shows `INFO` and above by default.
Handlers	Decide where messages go. A message must pass both the logger's level and the handler's.
`System.Logger`	A platform front end since Java 9 that routes to `java.util.logging` by default.
Security	Clean untrusted text before logging it, never log secrets, and keep logging libraries updated.

7. Try It

1. Replace the `System.out.println` debugging lines in one of your programs with `LOG.fine(...)`. How do you make them appear again without editing the code?
2. In `FileLogDemo`, remove the `setFormatter` line and run it. Open `payroll.log`. Why might a program want its log in this format?
3. Find a place in one of your labs where a value read from a file is printed or logged. Could a line break in that value cause a misleading line in the output?

8. Sources

Each title below is a link.

- The Apache Software Foundation, "Log4j Security Vulnerabilities" (logging.apache.org).
- Azul, "Do Apache Log4j CVE-2021-44228, CVE-2021-45046, or CVE-2021-45105 affect Azul's JVMs?" (support.azul.com), which states, "These CVEs do not affect the JDK directly."
- SEI CERT Oracle Coding Standard for Java, "IDS03-J. Do not log unsanitized user input" (wiki.sei.cmu.edu).
- OWASP, "Logging Cheat Sheet" (cheatsheetseries.owasp.org).
- OpenJDK, "JEP 264: Platform Logging API and Service" (openjdk.org).